

# HACK-ADEMY

*Tools for Hackathon Success*



# Content



Introduction



Understanding Optimisation



AI Strategies & Tips



Website Walkthrough



Hands-on Prac

# What is Entelect Hackathons?

## The short version:

Timed boxed coding competition

- One problem statement, endless possible solutions.
- Any programming language is welcome
- AI usage is embraced

## Everything runs through the hackathon website:

- Download the problem statement and inputs.
- Upload solution.
- Automatic scoring, in seconds.
- Live leaderboard.

## Where you will find them:

- Community hackathons – HackIT.
- Community workshops and events.
- University Cups.



# Optimisation Problems

Fundamental computational challenges used to teach and evaluate algorithm design. The best solution depends on constraints such as time, memory, and input size.

## Main Categories

### Dynamic Programming (DP)

- Solves overlapping subproblems by storing intermediate results.
- Examples: Knapsack, Coin Change, Longest Common Subsequence (LCS).

### Greedy Algorithms

- Makes the best local choice at each step to achieve an optimal overall solution.
- Examples: Activity Selection, Dijkstra's Shortest Path, Huffman Encoding.

### Graph Algorithms & Searching:

- Explores networks and relationships for navigation and optimisation.
- Examples: Travelling Salesman Problem (TSP), Depth-First Search (DFS), Breadth-First Search (BFS).



# Optimisation in Day to Day

**Driving from Sandton to O. R. Tambo International Airport.**

## Alternative Routes

Multiple routes are calculated simultaneously.

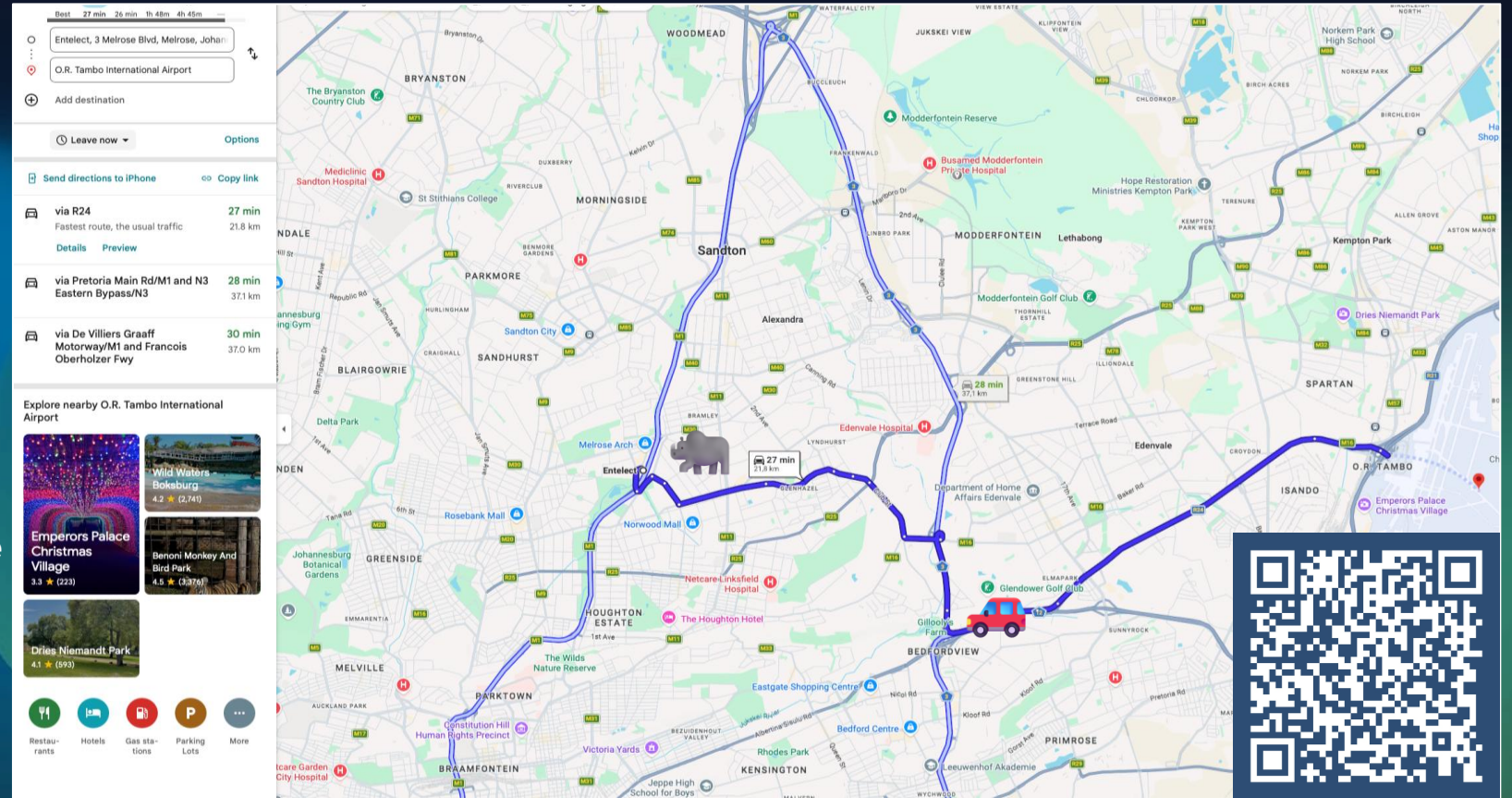
## Real-Time Updates

Google Maps continuously receives live traffic information.

## Route selection

Shortest path does not always mean fastest.

Highways vs local roads.





# Strategies

## Two approaches:

Know where to start

- Sketch out your idea.
- Use AI as a reviewer rather than a solution generator.

Unsure of where to start

- Summarise problem.
- Explain inputs & outputs.
- Identify constraints.
- Explain scoring.

## Divide and Conquer — only AFTER everyone is on the same page

- Everyone needs to understand the problem.
- Brainstorm approaches and agree on boilerplate code.
- Then divide and conquer.



# AI Tips

## Think alongside the model:

- Understand every Solution.
- Ask the model to generate documentation.
- Review generated code.
- Try to improve upon the AI's output.

## Prompt Well:

- Use structured markdown.
- Be detailed and provide all files and context.
- Ask the LLM to tell you if information is missing or if something is unclear.

## Iterate with Models and Solutions:

- Ask one LLM to help generate and refine the prompt for another.
- Feed the solution plan of one LLM to another model for revision.
- Try to improve upon the AI's output.
- Generate a few solutions and improve upon the best one.

## Pitfalls:

- Blindly trusting generated code.
- Not verifying assumptions.
- Spending hours prompt-engineering instead of testing.



# The Problem Statement

## Objective

What are you actually solving?

What does “done” look like?

Summarise it in one sentence before you code.

## Inputs

What data do I get?

What format is it in?

*Ask: how will I load this into my code?*

## Constraints

The rules of the game.

Limits, formats, and fine print.

Where edge cases hide.

Break them – your submission is invalid.

## Outputs

What exactly do you submit?

Match the required format precisely.

## Scoring

How are points calculated?

Optimise for the formula

## Hints

Examples and guidance.

Take it level by level





# Practice Problem: Ranger's Rescue Route

## The mission

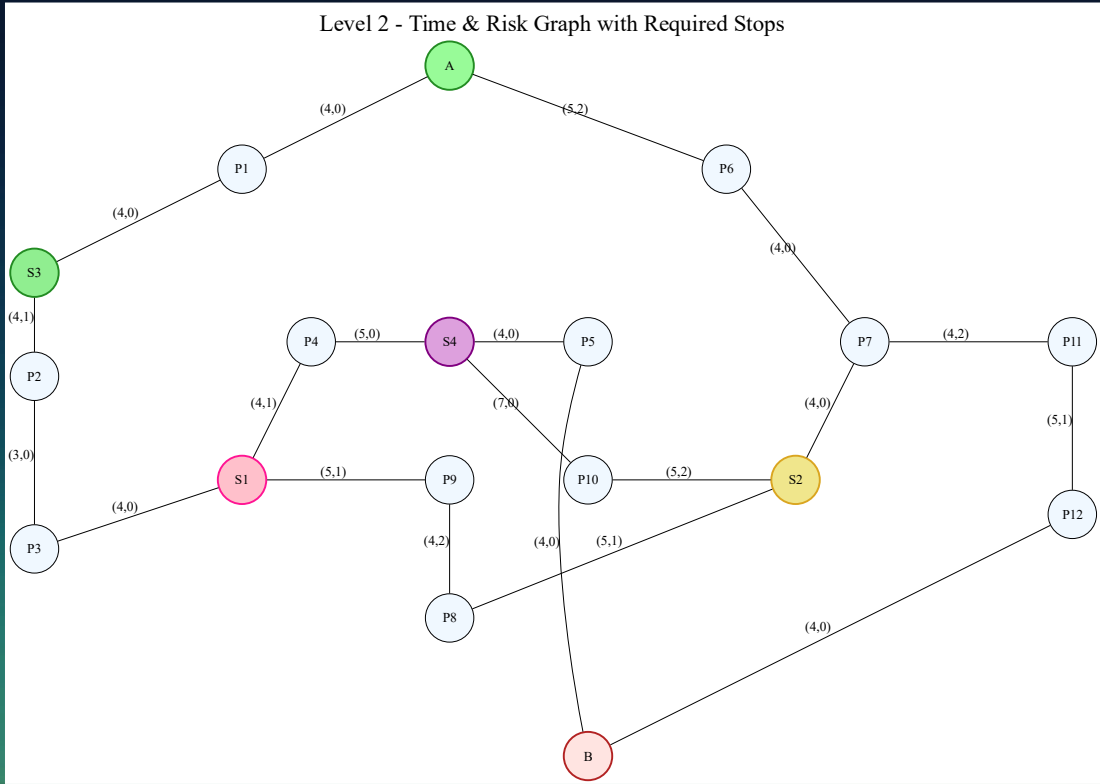
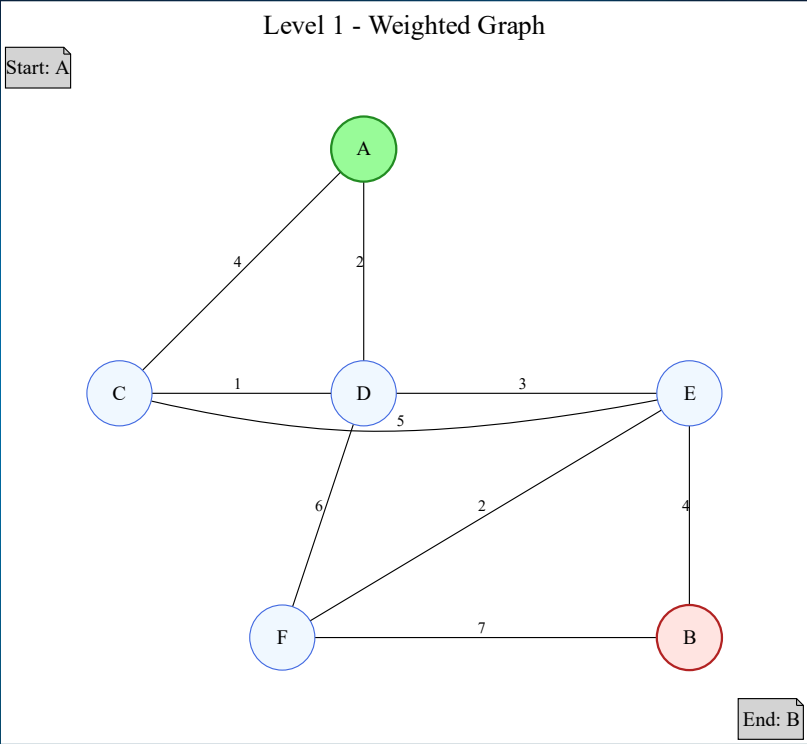
- You are a wildlife ranger. Find the cheapest route through a network of trails to reach a rescue.
- Objective:
  - Level 1: Quickest travel time from A to B
  - Level 2: Quickest travel with the least amount of risk from point A to B while making 4 separate stops to pick up some resources
- Input: the graph as an adjacency list – undirected, weighted.
- Output: { "route": ["A", "D", "E", "B"] } – node names in order.

## Scoring

- Each level is a score out of 100



# Practice Problem: Ranger's Rescue Route



# Website Walkthrough

## Using the Website

- Everything lives in one place: the problem statement, level inputs, submissions, and the leaderboard.
- Log in, register for a hackathon, and download the problem statement and level files on the day.
- Read first, code later: know the objective, format, and scoring before you write a line.
- When you are ready, head to the submissions page.
- Let's click through it together!



# Website Walkthrough

## Submitting & Getting a Score

Upload two things:

- Your answer file (.txt or .json)

```
{  
  "route": ["A", "D", "E", "B"]  
}
```
- Your source code (.zip).
  - Include a README in case a reviewer has to run your code.

Your score comes back instantly:  $\text{optimal cost} \div \text{your cost} \times 100$ .



# Website Walkthrough

## Iterate, Iterate, Iterate

Your first submission is a baseline, not your final answer.

A perfect route on paper scores zero if it never gets submitted.

The loop:

- Submit early. Even a naive route earns points.
- Measure. Compare your score against the maximum (when available).
- Improve one thing, submit again, and watch the number climb.





# Resources

---

Scan the QR code:

- Prompt guide.
- Problem statement
- Solution guide (pending)
- Slides

Happy Hacking!



**Scan Me**